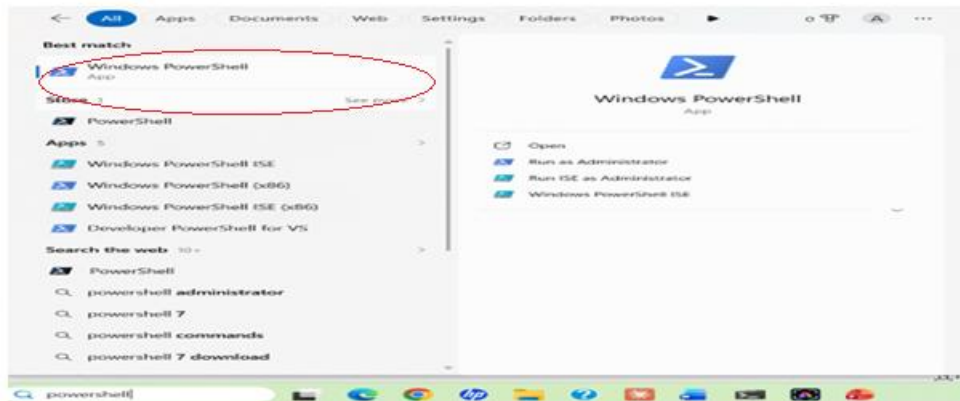


# SKILL 1

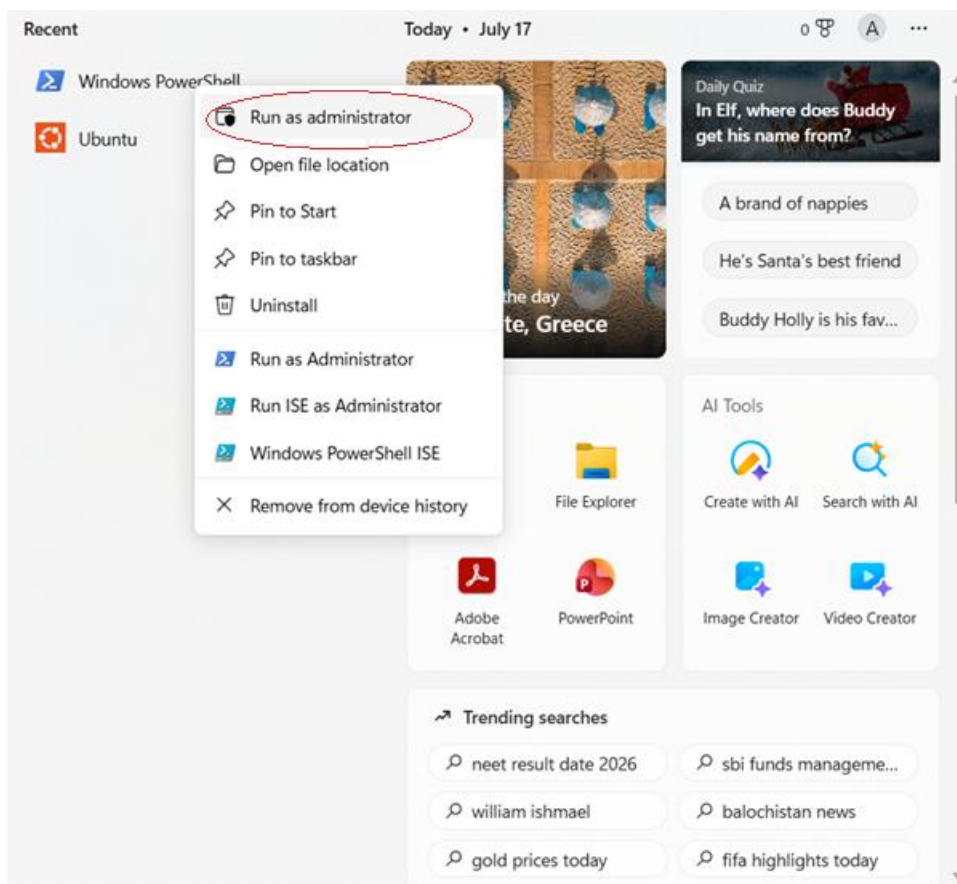
## Wsl installation-

### Step 1: Open PowerShell as Administrator

1. Press **Windows**.
2. Search for **PowerShell**.



3. Right-click **Windows PowerShell**.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\windows\system32> wsl --install
```

### Step 3: Restart Your Computer

After installation, restart your PC when prompted.

### Step 4: Complete Ubuntu Setup

After restarting:

1. Ubuntu opens automatically (or search for **Ubuntu** in the Start menu).
2. Wait for the installation to finish.
3. Create a Linux username.

Example:

Enter new UNIX username: anusha

### Step 5: Create a Password

You'll then be prompted to create a password.

New password: \*\*\*\*

Retype new password:\*\*\*\*

### Step 6: Update Ubuntu

```
manasvi@ManasviCG:~$ sudo apt update
[sudo: password for manasvi:]
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [293 kB]
Hit:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [357 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [76.5 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [31.1 kB]
Get:9 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [91.1 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [138 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [53.1 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [133 kB]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [43.9 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [235 kB]
Get:16 http://security.ubuntu.com/ubuntu resolute-security/universe-amd64 Components [43.1 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/restricted-amd64 Components [47.6 kB]
Get:18 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [45.5 kB]
Fetched 2149 kB in 7s (295 kB/s)
74 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

### Step 7: Install GCC (C Compiler)

```
manasvi@ManasviG:~$ sudo apt install build-essential
Installing:
  build-essential

Installing dependencies:
  bzip2                gcc-x86-64-linux-gnu      libisl23
  cpp                  libalgorithm-diff-perl   libitm1
  cpp-15               libalgorithm-diff-xs-perl liblsan0
  cpp-15-x86-64-linux-gnu libalgorithm-merge-perl libmpc3
  cpp-x86-64-linux-gnu libasan8                  libquadmath0
  dpkg-dev             libc-dev-bin             libstdc++-15-dev
  fakeroot             libc6-dev                libtsan2
  g++                  libc6-1-0                libubsan1
  g++-15               libcrypt-dev             linux-libc-dev
  g++-15-x86-64-linux-gnu libdpkg-perl              lto-disabled-list
  g++-x86-64-linux-gnu libfakeroot               make
  gcc                  libfile-fcntllock-perl  manpages-dev
  gcc-15               libgcc-15-dev            rpcsvc-proto
  gcc-15-base          libgomp1
  gcc-15-x86-64-linux-gnu libhwasan0

Suggested packages:
  bzip2-doc            gcc-15-doc               gcc-doc
  cpp-doc              gcc-multilib             gcc-15-multilib
```

### Step 8: Verify the Installation

```
manasvi@ManasviG:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

### Step 9: Create Your C Program

Example: nano process.c

### Step 10: Compile the Program

```
manasvi@ManasviiG:~$ gcc process.c -o process
manasvi@ManasviiG:~$ ./process
Enter a Linux command: pwd

Parent Process
Parent PID   : 1876
Child PID    : 1877

Child Process
Child PID    : 1877
Parent PID   : 1876
/home/manasvi

Child process completed.
```

### Install and configure GCC for compiling C programs.

1. Open the Ubuntu Terminal.

2. Update the package list:

```
sudo apt update
```

3. Install GCC:

```
sudo apt install gcc
```

4. Check the GCC version:

```
gcc --version
```

5. Create a C file:

```
nano hello.c
```

6. Write the C program:

```
#include <stdio.h>
```

```
int main(){
    printf("Hello, Ubuntu!\n");
    return 0;
}
```

7. Save the file using Ctrl + O → Enter, then exit using Ctrl + X.

8. Compile the program:

```
gcc hello.c -o hello
```

9. Run the program:

```
./hello
```

10. If you get Hello, Ubuntu!, GCC is successfully installed and working.

## Create a project folder and initialize it as a Git repository using git init.

1. Open the Ubuntu Terminal.
2. Create a project folder:

```
mkdir os-project
```

3. Enter the project folder:

```
cd os-project
```

4. Initialize the folder as a Git repository:

```
git init
```

5. Check the Git repository status:

```
git status
```

6. If successful, you will see something like:

```
On branch master/main
```

```
No commits yet
```

This means the project folder has been successfully converted into a Git repository.

❖ **Create folders such as src, bin, docs, and tests.**

❖ **Create basic project files like README.md, Makefile, and main.c.**

1. Go to your project folder:

```
cd os-project
```

2. Create the required folders:

```
mkdir src bin docs tests
```

3. Create the README.md file:

```
touch README.md
```

4. Create the Makefile:

```
touch Makefile
```

5. Create the main.c file inside the src folder:

```
touch src/main.c
```

6. Check the project structure:

```
ls
```

7. To see everything inside the folders:

```
find .
```

❖ **Check the current shell using echo \$SHELL and view the shell process using ps.**

```
manasvi@ManasviG:~$ echo $SHELL
/bin/bash
manasvi@ManasviG:~$ ps
  PID TTY          TIME CMD
 3581 pts/2    00:00:00 bash
 3716 pts/2    00:00:00 ps
manasvi@ManasviG:~$ |
```

1. Create a simple C program inside the src folder.
2. Create a Makefile with GCC compilation commands.

```
manasvi@ManasviG:~$ nano skilll.c
manasvi@ManasviG:~$ gcc skilll.c
manasvi@ManasviG:~$ ./a.out
Hello, OS Project!
manasvi@ManasviG:~$ |
```

```
manasvi@ManasviG:~$ nano Makefile
manasvi@ManasviG:~$ make
gcc src/main.c -o main
cc1: fatal error: src/main.c: No such file or directory
compilation terminated.
make: *** [Makefile:4: all] Error 1
manasvi@ManasviG:~$ make ./main
make: *** No rule to make target 'main'.  Stop.
manasvi@ManasviG:~$ |
```

```
manasvi@ManasviG:~$ make clean
rm -f main
manasvi@ManasviG:~$ |
```